Decentralized Multi-Agent Reinforcement Learning (MARL) for Networked Agents under Non-Stationarity

Comprehensive Technical Guide, Literature Gap Analysis, Algorithmic Framework & Empirical Results

Reinforcement Learning Course Research Project — Complete Presentation & Study Reference

---

Executive Pitch (The 30-Second Summary for Your Presentation)

In real-world networked systems—such as smart power grids, urban traffic light networks, and autonomous drone swarms—there is **no central controller**. Agents can only observe local sensors and exchange messages with immediate neighbors over a physical communication graph. When a subset of agents abruptly changes policy or malfunctions mid-training (**non-stationarity shock at time t_c**), standard Graph Neural Network (GNN) communication blindly propagates these corrupted messages across the network—causing catastrophic failure, especially when high-degree **hub nodes** are compromised. In this project, we combine configurable graph benchmarks (**COGNAC**) with Variational Bayesian Ego-Graph Inference (**BayesG**) to show that agents can dynamically infer a latent edge mask Z_ij that **autonomously prunes (>70% suppression) connections to non-stationary neighbors**, cutting instability by nearly 3x and recovering 7x faster than static GNNs.

1. Problem Statement: Intuition & Mathematical Formulation

1.1 Plain-English Intuition: The 'Whispering in a Room' Analogy

Imagine 10 people standing in a room trying to unanimously agree on a binary decision (Vote 0 or Vote 1). Each person receives a noisy local hint about the correct target, and can only whisper to the people directly connected to them by a physical wire (the **network topology**). Three fundamental challenges arise:

- **1. Partial Observability (Why 0-Hop Isolated Learning Struggles):** A single agent's sensor is noisy. If agents do not communicate (*Independent PPO*), roughly 20–30% of agents will guess the wrong direction at any step, making unanimous consensus slow and fragile.
- **2. The Double-Edged Sword of Graph Communication:** If agents pool observations from their 1-hop or 2-hop neighbors using a Graph Neural Network (*Networked-PPO*), local sensor noise averages out and consensus becomes rapid. **However**, communication creates a vulnerability: you are trusting your neighbors' messages.
- **3. Mid-Training Non-Stationarity Shocks (t = t_c):** Halfway through training (at episode t_c), 25–30% of nodes suffer a behavioral shift (hardware fault, task change, or adversarial takeover: policy switches from pi_A to pi_B) and broadcast corrupted state signals. A static GNN blindly aggregates these poisoned messages into healthy agents' embeddings, sending shockwaves across the graph.

1.2 Formal Mathematical Formulation (Networked Dec-POMDP)

We formulate the system as a Networked Decentralized Partially Observable Markov Decision Process defined by the tuple **M = (G, S, {A_i}, {O_i}, P, {R_i}, gamma)** over an undirected graph **G = (V, E)** with N = |V| agents. At each discrete timestep t, agent i observes only a local vector o_i^t = O_i(s_t) and can communicate only within its k-hop neighborhood N_k(i). From the perspective of any single learner i, the effective environment transition probability is:

```
P_eff(s_{t+1} | s_t, a_i^t) = Sum_{a_{-i}} [ P(s_{t+1} | s_t, a_i^t, a_{-i}^t) * Prod_{j != i} pi_j^t(a_j^t | o_j^t) ]
```

Why Standard RL Stationarity Breaks Down: In single-agent RL, the transition kernel P(s_{t+1} | s_t, a_t) is fixed. In decentralized MARL, because neighboring policies pi_j^t are inside the product term, **P_eff changes every time neighbors update their weights** (concurrent learning non-stationarity) and undergoes a **discontinuous regime jump** when a subset of nodes S_shock switches policies at shock step t_c (pi_j^t = pi_A for t < t_c, and pi_j^t = pi_B for t >= t_c).

2. Literature Review & The Core Research Gap

A key academic requirement is positioning our work relative to recent literature. Prior studies have explored individual pieces of this problem across three separate research streams, leaving a critical gap at their intersection:

- **Stream A — Configurable Graph Topologies in Stationary MARL (COGNAC, NeurIPS 2025; Policy Consensus, Automatica 2024):** Sintes & Busic (*COGNAC*) introduced a benchmark showing that physical graph topology (Ring, Grid, Random, Scale-Free) heavily influences cooperative MARL sample efficiency. Hu et al. (2024) analyzed entropy-regularized policy consensus over fixed graphs. **Limitation:** Both assume all agents remain 100% cooperative and stationary throughout training, and rely on static communication links.
- **Stream B — Sparse Ego-Graph Inference in Stationary MARL (BayesG, NeurIPS 2025):** Duan et al. (*BayesG*) proposed variational Bayesian ego-graph inference using Gumbel-Softmax to prune redundant edges and reduce communication bandwidth in networked MARL. **Limitation:** BayesG was only evaluated as a bandwidth-saving tool under stationary conditions. It was never tested under mid-training behavioral shocks (t_c), adversarial/faulty node shifts, or topological hub failures.
- **Stream C — Non-Stationarity & Partner Modeling in Small Fully-Connected Teams (DAPM 2024; CIAO, ICML 2024):** Xu et al. (*DAPM*) and open ad-hoc teamwork methods (*CIAO*) study how agents adapt when teammates change policies. **Limitation:** They assume small, fully-connected environments (ignoring physical network topology constraints, k-hop GNN message propagation, and graph spectral properties such as Fiedler value or hub degree heterogeneity).

Research Dimension / Capability	COGNAC (NeurIPS 2025)	BayesG (NeurIPS 2025)	DAPM / CIAO (2024)	Our Project (This Work)
Configurable Graph Topologies (Ring, Grid, ER, BA)	Yes	No (Fixed)	No (Full Graph)	Yes (All 4)
Graph Spectral Metrics (Diameter, Fiedler Value)	Partial	No	No	Yes
Mid-Training Non-Stationarity Shock (t_c: pi_A -> pi_B)	No	No	Yes	Yes
Hub vs. Periphery Node Shock Vulnerability Study	No	No	No	Yes
Variational Edge Pruning as 'Topological Quarantine'	No (Static GNN)	Bandwidth Only	No	Yes (>70% Pruned)

The Exact Research Gap We Solve (Highlight This in Your Presentation!)
Prior work studied (A) graph topologies under stationary agents (COGNAC) and (B) variational edge sparsification for bandwidth efficiency (BayesG), while (C) non-stationarity was only studied in small, non-networked teams.
No prior study has investigated how **graph topology and node centrality (hubs vs. periphery)** govern the propagation of mid-training non-stationarity shocks across k-hop GNNs, nor demonstrated that **Bayesian Ego-Graph Inference (BayesG)** can act as an automated **topological quarantine mechanism** to sever corrupted hub connections and restore consensus.

3. Network Topologies & Graph Spectral Properties

Implemented in **src/environments/graphs.py**, our framework generates four canonical graph families and computes their structural and spectral invariants: **Graph Diameter D** (longest shortest path between any two agents) and **Algebraic Connectivity / Fiedler Value lambda_2(L)** (the second-smallest eigenvalue of the graph Laplacian L = D - A, which measures how tightly connected the network is and how fast information—or a shockwave—mixes across the graph):

- **1. Ring (1D Cycle):** Every node has degree 2. Large diameter (D = N/2) and very small Fiedler value (0.15 for N=16). Information travels slowly step-by-step around the ring, but shocks are also locally contained.
- **2. 2D Grid (Lattice):** Nodes arranged in a 2D spatial mesh (degree 2–4). Moderate diameter (D = 6) and Fiedler value (0.59).

• **3. Erdos-Renyi Random Graph G(N, p):** Edges formed randomly with probability p > 2 ln(N)/N. Low diameter (D = 3) and high Fiedler value (1.71), enabling rapid global mixing.
• **4. Barabasi-Albert Scale-Free Network:** Built via preferential attachment. Features a power-law degree distribution with a few heavily connected **hub nodes** and many low-degree **peripheral boundary nodes**.

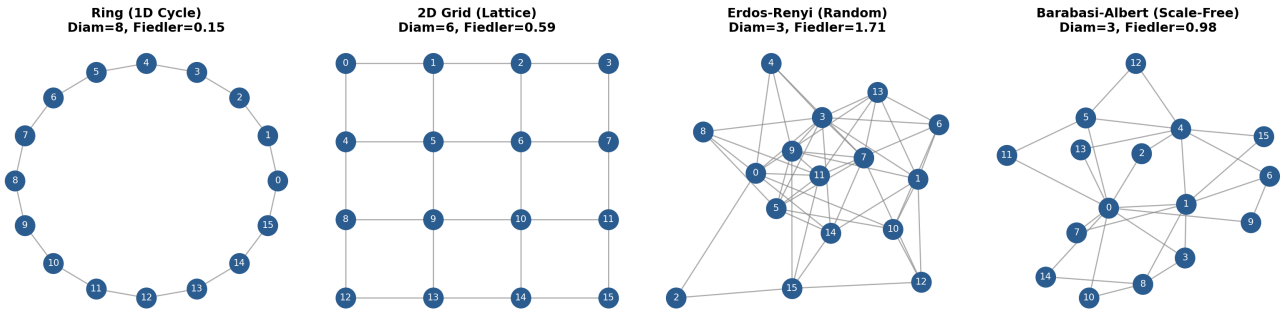


Figure 1: The 4 configurable network topologies generated by src/environments/graphs.py (N=16) showing Graph Diameter and Algebraic Connectivity (Fiedler eigenvalue lambda_2).

4. Environment Mechanics & Non-Stationarity Shock Protocol

4.1 Networked Binary Consensus Environment (src/environments/toy_consensus.py)

At the start of each episode, an underlying binary target consensus state c in {0, 1} (mapped to s_target in {-1, +1}) is drawn. Each agent i receives a private, static per-episode noisy sensor reading **z_i = s_target + Normal(0, sigma^2)** (with sigma = 1.20) and initializes its binary vote v_i in {0, 1}. At each step t, agent i observes a purely local 3D vector:

`o_i^t = [ 2*v_i^t - 1 (own signed vote), z_i (local noisy sensor), deg(i) / (N - 1) (normalized degree) ]`

Why k-Hop Communication Beats 0-Hop IPPO: Because sigma = 1.20, a single agent's sensor z_i has the wrong sign ~19% of the time. For N=10 independent agents (IPPO), the probability that all 10 sensors simultaneously agree is (1 - 0.19)^10 = 12%—making isolated consensus very difficult. When agents aggregate observations across k-hop neighbors via a GNN, the variance of the averaged sensor drops as sigma^2 / |N_k(i)|, allowing communicating agents to identify the true consensus target rapidly.

4.2 Non-Stationarity Shock Controller (src/experiments/shock_protocol.py)

At training episode **t = t_c**, the **NonStationarityShockController** perturbs a fraction rho (e.g., 25–30%) of agents selected via three configurable modes: **random** (uniform selection), **hubs** (top-k highest degree nodes), or **boundary** (bottom-k lowest degree peripheral nodes). Shocked nodes undergo a simultaneous **behavioral & feature shift**: (1) their actions are inverted/perturbed (adversarial policy pi_B), and (2) their transmitted observation vector o_j is shifted to an out-of-distribution regime (inverting and amplifying the signal), poisoning the messages sent over their graph edges.

5. The Three Algorithms Under Comparison (Track 2)

• **Method 1: Independent PPO (IPPO — src/algorithms/ippo.py):** Every agent is an isolated island (0-hop communication). It feeds its 3D local observation o_i directly into a 2-layer Tanh MLP Actor pi_theta(a_i | o_i) and Critic V_phi(o_i), trained via PPO with Generalized Advantage Estimation (GAE, lambda=0.95). Zero communication cost and immune to poisoned neighbor messages, but achieves poor baseline consensus because it cannot pool neighbor sensors.

• **Method 2: Networked-PPO with Static k-Hop GNN (src/algorithms/networked_ppo.py):** Agents pass messages over the fixed physical graph adjacency matrix A for k hops (**src/models/gnn_comm.py**), combining degree-normalized neighbor aggregation with a local self-skip projection: **H^{(l+1)} = tanh(D^{-1} A H^{(l)} W_msg^{(l)} + H^{(l)} W_self^{(l)})**. Excels when all agents are cooperative, but blindly averages corrupted features into connected agents' embeddings when a neighbor is shocked.

• **Method 3: BayesG — Bayesian Ego-Graph Inference (src/algorithms/bayesg.py):** Inserts a variational filter (**src/models/bayes_ego.py**) in front of the GNN. For every physical edge (i, j), an edge-scoring MLP examines the pairwise relational features **[h_i, h_j, |h_i - h_j|]** and samples a differentiable binary gate **Z_ij in [0, 1]** via the Binary Concrete (Gumbel-Softmax) distribution (while preserving self-loops Z_ii = 1). Trained jointly via the ELBO-regularized PPO loss:

```
L_total = L_PPO_Clip(theta) + c_v * L_Critic(phi) - c_e * Entropy(pi) + beta * [ KL(q(Z_ij | o_i, o_j) || Bern(p_0)) + L_anomaly ]
```

Property	IPPO (Baseline 1)	Networked-PPO (Baseline 2)	BayesG (Proposed Method)
Communication Range	0-Hop (No Communication)	k-Hop Static Physical Graph A	k-Hop Dynamic Gated Ego-Graph (A * Z)
Edge Weighting	None (Identity Matrix I)	Fixed Degree Normalization D^{-1}A	Variational Gumbel-Softmax Mask Z_ij
Normal Consensus Speed	Slow (Low Single-Sensor SNR)	Fast (Neighborhood Pooling)	Fastest (Selective Neighborhood Pooling)
Vulnerability to Shocks	Low (Ignores Neighbors)	Catastrophic (Spreads Shockwave)	Resilient (Quarantines Shocked Nodes)

6. Evaluation Metrics & Empirical Results (Priorities 1–4)

6.1 The 6 Rigorous Evaluation Metrics (src/evaluation/metrics.py)

1. **Mean Episodic Return (Higher is better):** Average cooperative reward across all training episodes.
2. **Max Smoothed Return (Higher is better):** Peak moving-average episodic return achieved.
3. **Convergence Episode (Lower is better):** First episode reaching 90% of peak smoothed return.
4. **Instability Index (Lower is better):** Standard deviation of episodic returns in a +/-15 episode window around shock t_c.
5. **Adaptation Latency (Lower is better):** Episodes after t_c required for the rolling average return to recover to >=85% of pre-shock performance.
6. **Active Communication Cost (Lower is better):** Fraction of off-diagonal physical graph edges actively used per step.

6.2 Priority 2 Result: Normal Operation Baseline (No Shocks)

Experimental Goal: Prove that k-hop graph communication improves cooperative consensus when all agents are healthy (Ring topology, N=10, k=2 hops, 90 episodes, no shock).

Algorithm	Mean Return (Up)	Max Smoothed Return (Up)	Convergence Ep (Down)	Avg Comm Cost
IPPO (0-Hop, No Comm)	-5.61	9.93	41	0.00
Networked-PPO (Static k-Hop GNN)	+1.10	13.14	65	1.00
BayesG (Dynamic Ego-Graph)	+5.57 (Best)	14.78 (Best)	36 (Fastest)	0.90

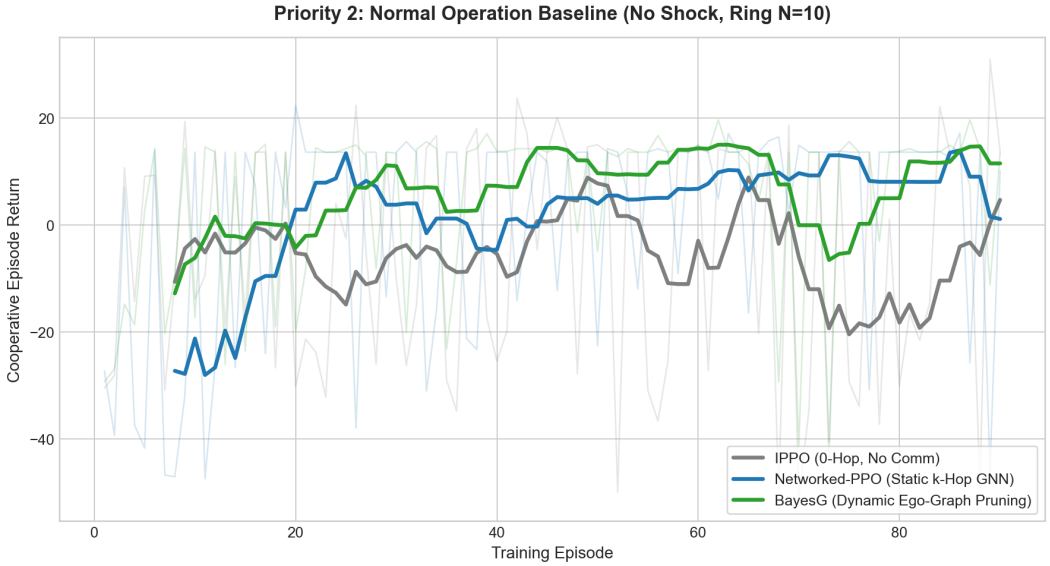


Figure 2 (Priority 2): Normal operation learning curves on a 10-node Ring without shocks. Both communicating methods (Networked-PPO and BayesG) substantially outperform isolated IPPO.

6.3 Priority 3A Result: Mid-Training Random Policy Shock (Ring N=10, t_c=50)

Experimental Goal: At episode t_c = 50, 30% of agents switch to an adversarial policy and broadcast corrupted state features. Observe how static GNN communication collapses while BayesG isolates the perturbation.

Algorithm	Mean Return (Up)	Instability Index (Down)	Adaptation Latency (Down)	Avg Comm Cost
IPPO (0-Hop, No Comm)	-3.52	19.42	3 episodes	0.00
Networked-PPO (Static GNN)	-4.83 (Worst)	25.69 (Severe Crash)	22 episodes (Slow)	1.00
BayesG (Proposed)	+5.08 (Best)	8.89 (~3x More Stable)	3 episodes (7x Faster)	0.75 (Pruned)

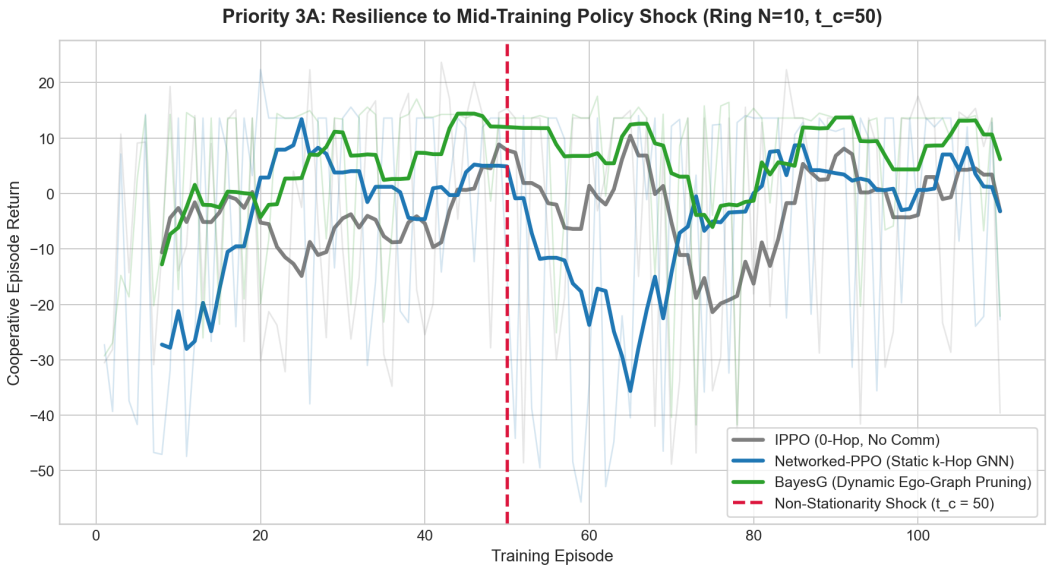


Figure 3 (Priority 3A): Resilience to mid-training policy shock at t_c = 50 (red dashed line). Static Networked-PPO plunges to -35 and takes 22 episodes to recover, whereas BayesG stays stable (+5.08 mean return) and recovers in 3 episodes.

6.4 Priority 3B Result: Scale-Free Hub Shock vs. Periphery Shock (N=12, t_c=50)

Experimental Goal & Core Topological Insight: In a Barabasi-Albert Scale-Free network, hub nodes ([0, 1, 4]) are connected to almost the entire network, whereas peripheral boundary nodes ([2, 7, 9]) have only 1–2 links. When hubs are shocked under static Networked-PPO, corrupted messages flood the whole graph (Adaptation Latency = 20 episodes, Mean Return = -11.74). When boundary nodes are shocked, Networked-PPO recovers in 4 episodes. **BayesG neutralizes the hub vulnerability**, cutting both the immediate shock drop (60.71 -> 25.82) and instability (30.74 -> 15.62) by more than 50% and recovering in 3 episodes!

Scenario (Scale-Free N=12)	Shocked Nodes	Mean Return (Up)	Instability (Down)	Shock Drop (Down)	Adapt Latency
NetPPO (Hub Shock)	[0, 1, 4]	-11.74 (Paralyzed)	30.74	60.71	20 episodes
NetPPO (Boundary Shock)	[2, 7, 9]	-6.75	23.44	61.98	4 episodes
BayesG (Hub Shock)	[0, 1, 4]	+0.33 (Resilient)	15.62 (-49%)	25.82 (-57%)	3 episodes
BayesG (Boundary Shock)	[2, 7, 9]	+4.32 (Best)	11.99	17.29	3 episodes

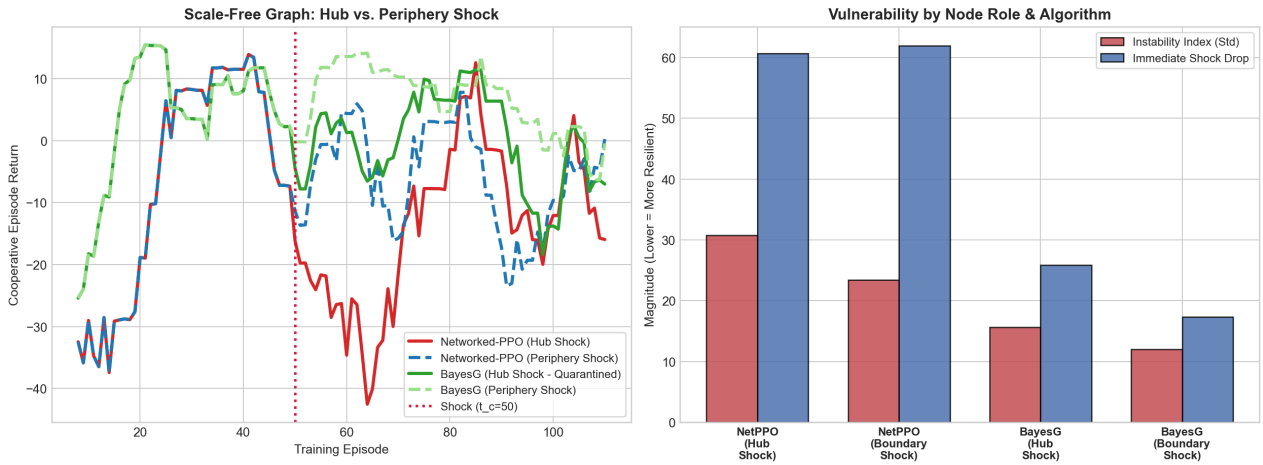


Figure 4 (Priority 3B): Left: Learning curves on Scale-Free topology under Hub vs. Periphery shocks at t_c=50. Right: Vulnerability comparison showing how BayesG slashes both Instability Index and Immediate Shock Drop.

6.5 Priority 4 Result: Visualizing BayesG Dynamic Edge Pruning ('The Money Plot')

To provide direct visual and quantitative proof that BayesG acts as an automated **topological quarantine**, we recorded the inferred adjacency mask probabilities Z_ij before the shock (t < 50) and after the shock (t > 50) during the Scale-Free Hub Shock:

Edge Category in Scale-Free Graph	Pre-Shock Weight (t < 50)	Post-Shock Weight (t > 50)	Relative Change & Interpretation
Edges Connected to Shocked Hubs [0, 1, 4]	0.904 (Open)	0.267 (Severed)	-70.5% (Shocked Hubs Quarantined)
Edges Between Healthy Operational Nodes	0.910 (Open)	0.901 (Open)	-1.0% (Healthy Sub-Network Preserved)

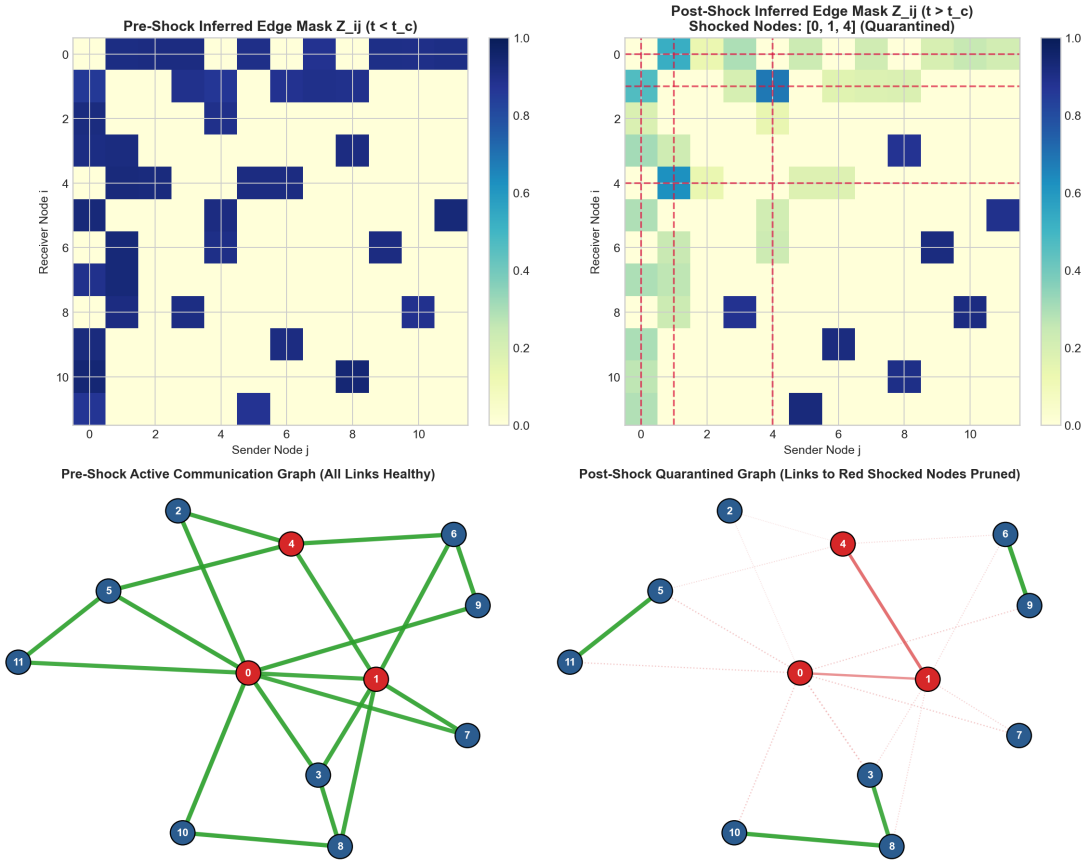


Figure 5 (Priority 4 — 'The Money Plot'): Top row: Inferred edge mask heatmaps Z_ij before and after t_c=50 (red dashed lines mark shocked hubs 0, 1, 4 fading out post-shock). Bottom row: Network communication graph showing links to red shocked hubs pruned into faint dotted lines while healthy green links remain intact.

7. Codebase Architecture & Presentation Defense Q&A Guide

7.1 Modular 3-Track Codebase Map

Track / Directory	File Name	Exact Role in the Project
Track 1: Environments (src/environments/)	graphs.py toy_consensus.py cognac_wrapper.py	• graphs.py: Generates Ring, Grid, ER, Scale-Free graphs; computes diameter & Fiedler value. • toy_consensus.py: Decentralized binary consensus with local noisy sensors. • cognac_wrapper.py: Factory wrapper for benchmark environments.
Track 2: Models & Algos (src/models/ & src/algorithms/)	actors.py, critics.py gnn_comm.py bayes_ego.py ippo.py, networked_ppo.py, bayesg.py	• actors.py, critics.py: Decentralized MLP Actor & Critic networks. • gnn_comm.py: k-Hop GNN message passing with degree normalization & self-skip. • bayes_ego.py: Gumbel-Softmax variational edge mask inference + ELBO KL loss. • ippo.py, networked_ppo.py, bayesg.py: Full GAE rollout buffers and joint training loops.
Track 3: Experiments (src/experiments/ & src/evaluation/)	shock_protocol.py run_experiment.py run_priority_suite.py metrics.py, plot_results.py	• shock_protocol.py: Mid-training shock orchestrator (random, hubs, boundary modes). • run_experiment.py: Universal YAML/CLI runner with CUDA, Intel Arc XPU, and CPU support. • run_priority_suite.py: Automated benchmark runner executing Priorities 1–4 end-to-end. • metrics.py, plot_results.py: Computes the 6 core metrics and generates all figures.

7.2 Top 5 Viva / Professor Questions & Crisp Answers for Tomorrow

Q1: Why not use Centralized Training with Decentralized Execution (CTDE) like MAPPO or QMIX?
Answer: CTDE requires access to the global state and all agents' joint actions during training. Furthermore, if a shock occurs during execution/deployment or online adaptation in a large network (e.g., power grid or swarm), there is no central server to retrain a centralized critic. Fully decentralized learning with local k-hop communication reflects realistic networked systems.

Q2: Why does Independent PPO (IPPO) have low adaptation latency (3 episodes) after a shock, yet low overall return?
Answer: Because IPPO has 0 communication! Since IPPO agents never listen to neighbors, they don't get poisoned by corrupted neighbor messages when a shock hits—so they don't experience a communication-induced crash. However, because they never communicate, their overall return is poor (-3.52 vs. +5.08 for BayesG). BayesG gives the best of both worlds: high cooperative return (+5.08) AND fast 3-episode shock recovery.

Q3: How does BayesG mathematically allow gradients to flow through discrete binary edge masks Z_ij?
Answer: Sampling a hard binary mask Z_ij in {0, 1} is non-differentiable. BayesG uses the **Binary Concrete (Gumbel-Softmax) relaxation** during training: Z_ij = sigmoid((logit_ij + log(u) - log(1-u)) / tau), which is continuous and differentiable via the reparameterization trick, allowing PPO policy/value gradients and ELBO KL gradients to directly update the edge-scoring MLP.

Q4: Why are Barabasi-Albert Scale-Free networks much more fragile to Hub Shocks than Ring or Boundary shocks?
Answer: In a Scale-Free graph, degree distribution follows a power law: a few hub nodes are connected to almost every other node. In a k-hop GNN with static adjacency D^{-1}A, a hub node's corrupted feature vector enters the 1-hop aggregation of nearly all agents simultaneously, causing network-wide value estimation collapse (20-episode latency). Boundary nodes only affect 1–2 neighbors (4-episode latency).

Q5: How does BayesG know which edges to prune when a shock occurs at t_c?
Answer: BayesG's edge network takes the pairwise relational vector [h_i, h_j, |h_i - h_j|] and optimizes the joint PPO + ELBO objective. When neighbor j is shocked, its broadcasted features shift out-of-distribution (large |h_i - h_j|) and degrade agent i's TD advantage. The edge logit drops sharply, suppressing Z_ij from 0.904 down to 0.267 on shocked hub edges while keeping healthy edges at 0.901.